

NFA reduction using simulation relation

Samuel Lupták*

Abstract

The problem of reducing deterministic finite automata to their minimal form is a well researched one. In some cases, it may be beneficial to try and reduce their non-deterministic versions, although this problem is not as easy and straight forward. The problem can be solved by creating a simulation relation between states which is then used for the reduction. The goal of this paper is to implement an algorithm for computing simulation relations into the Mata library developed at the VeriFit research group. Mata already has a complex algorithm that computes simulations but we believed a less complex one might yield better results for some NFAs. Despite our best efforts, the results were quite underwhelming, showing minimal improvements over the already implemented algorithm. These experimental result only show the strength of the already implemented algorithm, considering other works that tried to surpass this algorithm.

Keywords: Nondeterministic automata — Reduction — Simulation relation

Supplementary Material: [Mata library](#) — [Implementation](#) — [Benchmark](#)

*xluptas00@stud.fit.vutbr.cz, Faculty of Information Technology, Brno University of Technology

1. Introduction

Deterministic finite automata (DFA) are a formal mathematical model used in many applications today, including formal verification, compilers, pattern matching and even fields beyond informatics such as biology. In practical applications, determinism is a well-needed property of these automata because non-deterministic finite automata (NFA) are harder, sometimes even impossible, to implement. In contrast, when designing a finite automaton, it is easier to construct a NFA because it allows more freedom in its construction and its size can be a lot smaller.

In some applications, like formal program verification, even NFAs may have an unpleasant size making determinization computationally hard, because of the potential explosion in the number of states of the final DFA [2]. In these situations, the reduction of NFA becomes useful. Unfortunately, NFA reduction is not as easy as DFA reductions. NFA reduction is proven to be a PSPACE complete problem [2], so the optimal solution is very hard to find. That is why heuristics are used instead, like construction of the simulation relation, to partially solve this problem. The main requirements for these heuristics are speed and the

number of reduced states. We have compared two algorithms that implement the same heuristic, so the number of reduced states will be the same, but speed will differ.

There already exists an algorithm inside Mata [1] that builds a simulation relation on NFAs that is based on a paper from F. Ranzato and F. Tapparo, who came up with a clever solution (abbreviated as *RT*) [5]. The final simulation relation is then used to reduce the number of states of a NFA. This reduction mechanism that takes the simulation relation and outputs the reduced automaton is already implemented and we have not changed this part.

The algorithm we implemented comes from a paper from L. Ilie, G. Navarro, and S. Yu (abbreviated as *INY*) [3]. Unfortunately, the algorithm given in this paper is not correct, so we present a corrected version. In contrast to the *RT* algorithm, this one is much simpler. The general idea was to implement this simpler algorithm to create the simulation relation and compare the two algorithms in terms of speed and search for automata with specific properties that we could compute the simulation faster on. Sadly, the results were not what we were expecting. Of the 26158 automata

we used for benchmarking, only in about 400 was the simulation computed faster, but only by a negligible amount of time.

If we take into account more works that tried to replace the already implemented solution inside Mata, which did not improve the time efficiency [4], we can safely say that the current solution is very strong and much more time and effort is needed to surpass this solution.

2. Preliminaries

Let us begin with some formal definitions that need to be understood so we can explain the algorithm we used in detail.

Definition 1 An *alphabet* is a non-empty finite set whose elements are called letters. It is denoted with the greek letter Σ .

Definition 2 A *string* is a sequence of letters $a_0a_1 \dots a_n$ over an alphabet Σ such that $\forall i \leq n : a_i \in \Sigma$. The empty string is denoted ϵ .

Definition 3 Let Σ be an alphabet, then Σ^* is the set of all strings over Σ . Every $L \subseteq \Sigma^*$ is called a *language* over Σ .

Next we define a non-deterministic finite automaton. We will use this definition extensively in the following text.

Definition 4 A Non-deterministic finite automaton M , denoted as *NFA* M , is defined as a quintuple $M = (Q, \Sigma, \delta, S_0, F)$ where:

- Q is a finite set of all states,
- Σ is an alphabet,
- $\delta : Q \times \Sigma \rightarrow \mathcal{P}(Q)$ is a transition function,
- $S_0 \subseteq Q$ is a set of all initial states,
- $F \subseteq Q$ is a set of all final states.

Definition 5 Let $M = (Q, \Sigma, \delta, S_0, F)$ be an NFA. The *Reverse NFA* M^R with respect to M is the quintuple $M^R = (Q, \Sigma, \delta^R, F, S_0)$ where δ^R is a function such that $\forall p, q \in Q : \forall a \in \Sigma : p \in \delta(q, a) \Leftrightarrow q \in \delta^R(p, a)$.

Notice that the only differences between the NFA M and the NFA M^R are that the initial and final states are swapped and all the transitions are reversed.

2.1 Reduction using equivalences

The basic idea in reducing the number of states of a NFA is to build an equivalence relation between the states. Let us define the properties that an equivalence relation must satisfy.

Definition 6 Let R be a binary relation. R is an *equivalence relation* if it is:

- **Reflexive** $\forall a : aRa$,
- **Symetric** $\forall a, b : aRb \Leftrightarrow bRa$,
- **Transitive** $\forall a, b, c : aRb \wedge bRc \implies aRc$.

Naturally, we need to specify when two states are in a relation.

Definition 7 Let $M = (Q, \Sigma, \delta, S_0, F)$ be an NFA. $\equiv_R$ is an *equivalence relation* which satisfies the following properties:

- $P_1 : \equiv_R \cap (F \times (Q - F)) = \emptyset$,
- $P_2 : \forall p, q \in Q : \forall a \in \Sigma : (p \equiv_R q \Rightarrow \forall q' \in \delta(q, a), \exists p' \in \delta(p, a), q' \equiv_R p')$,
- $P_3 : \forall p, q \in Q : p \equiv_R q \implies q \equiv_R p$

Firstly we define how we can merge states together and then give a definition on how to reduce automata using equivalences.

Definition 8 Let $M = (Q, \Sigma, \delta, S_0, F)$ be an NFA. When we *merge* states in M according to some partitioning P we construct a new NFA $M_m = (Q_m, \Sigma, \delta_m, S_{0m}, F_m)$:

- Q_m is the set of all partitions in P ,
- $(B, a, C) \in \delta_m$ if $(p, a, q) \in \delta$ for some $p \in B, q \in C$,
- S_{0m} is the set of partitions in P that contain at least one state from S_0 ,
- F_m is the set of partitions in P that contain at least one state from F .

Definition 9 Let $M = (Q, \Sigma, \delta, S_0, F)$ be an NFA. If we construct the largest $\equiv_R$ over M we can merge states according to the equivalence classes of $\equiv_R$ thus *reducing* the overall number of states.

Example 1 Let us give an example of a reduction using $\equiv_R$ and $\equiv_L$. Figure 1 is the unreduced automaton. Figure 2 and figure 3 are the reduced automata according to $\equiv_R$ and $\equiv_L$ respectively.

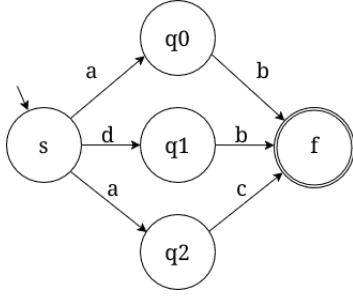


Figure 1. NFA M_1

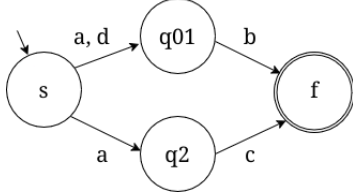


Figure 2. Reduced M_1 using $\equiv_R$

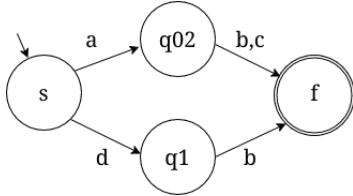


Figure 3. Reduced M_1 using $\equiv_L$

it would be the same as reducing with $\equiv_R$. The same holds for $\preceq_L$ using only the R2 rule, which equals to reduction with $\equiv_L$. The power of preorders comes from the R3 rule, which allows merging states in a way not possible with equivalences. We prove this in the following theorem.

Theorem 1 Let $M = (Q, \Sigma, \delta, S_0, F)$ be an NFA M . There exists an automaton M whose reduction with $\preceq_R$ and $\preceq_L$ will result in a lower amount of states in the reduced automaton than the reduction with $\equiv_R$.

Proof: We will prove this by construction.

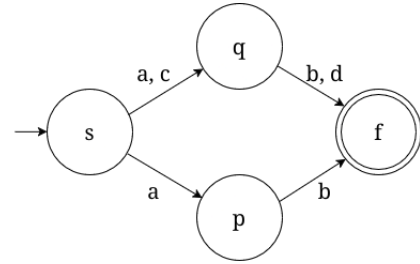


Figure 4. NFA M_2

If we construct $\equiv_R$ and $\equiv_L$ over M_2 depicted in Figure 4, we can see that no reduction is possible. But if we construct $\preceq_R$ and $\preceq_L$ we can see that $p \preceq_R q$ and $p \preceq_L q$ so we can use R3 to reduce this automaton. We can see that the reduced automaton in Figure 5 has less states than the automaton reduced by equivalences thus proving the strength of preorders.

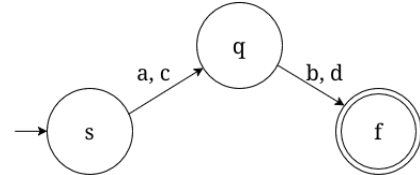


Figure 5. Reduced M_2 according to R3

From now on, we will refer to the preorder $\preceq_R$ and $\preceq_L$ as a *simulation*, $p \preceq_R q$ meaning that q simulates p .

2.3 Construction of the simulation

In this section we give the overview of the *INY* algorithm. As mentioned before, in the original paper, the algorithm contains an error, which has been corrected by Ondřej Lengál¹. Here, we show the corrected version of Algorithm 1. The algorithm computes the complement simulation which is then reversed on line 20. The fix is on line 7. The original algorithm did not include the second clause in the condition which is essential for the correct simulation computation.

¹<https://github.com/ondrik/in-y-fix>

2.2 Reduction using preorders

A much better reduction can be achieved if we use P_1 and P_2 from Definition 7 to construct a preorder instead of an equivalence relation. Let us define the properties that a preorder must satisfy.

Definition 10 Let R be a binary relation. R is a pre-order if it is:

- **Reflexive** $\forall a : aRa$,
- **Transitive** $\forall a, b, c : aRb \wedge bRc \implies aRc$.

With P_1 and P_2 we can construct a preorder $\preceq_R$ from the original automaton, as well as a preorder $\preceq_L$ from the reverse automaton.

Definition 11 Let $M = (Q, \Sigma, \delta, S_0, F)$ be an NFA. Let us construct the largest $\preceq_R$ and $\preceq_L$ over M and let P be a partitioning of Q . We can put together states into a partition in P if one of the following holds:

- $R1 : \exists p, q \in Q : q \preceq_R p \wedge p \preceq_R q$,
- $R2 : \exists p, q \in Q : q \preceq_L p \wedge p \preceq_L q$,
- $R3 : \exists p, q \in Q : q \preceq_R p \wedge q \preceq_L p$,

Thus we can merge states according to P and **reduce** the overall number of states.

The only difference between preorders and equivalences is that the latter is symmetric. If we construct only the relation $\preceq_R$ and reduce only by the R1 rule,

Algorithm 1: Computation of the maximum direct simulation $\preceq_R$

Input: NFA $\mathcal{A} = (Q, \Sigma, \delta, I, F)$ **Output:** Simulation $\preceq_R$ on $\mathcal{A}$

```
1 foreach  $q \in Q, a \in \Sigma$  do // preprocess
2 |   compute  $\delta^r(q, a)$ ;
3  $worklist \leftarrow$  empty;
4  $R \leftarrow \emptyset$ ;
5 foreach  $p \in Q, q \in Q, a \in \Sigma$  do // init
6 |    $cnt_a(p, q) \leftarrow |\delta(q, a)|$ ;
7 |   if  $(p \in F \wedge q \notin F) \vee (\delta(p, a) \neq$ 
8 |      $\emptyset \wedge \delta(q, a) = \emptyset)$  then
9 |      $R \leftarrow R \cup \{(p, q)\}$ ;
9 |      $worklist.enqueue((p, q))$ ;
10 while  $worklist \neq$  empty do // propagate
11 |    $(p', q') \leftarrow worklist.dequeue()$ ;
12 |   foreach  $a \in \Sigma$  do
13 |     foreach  $q \in \delta^r(q', a)$  do
14 |        $cnt_a(p', q) \leftarrow cnt_a(p', q) - 1$ ;
15 |       if  $cnt_a(p', q) = 0$  then
16 |         foreach  $p \in \delta^r(p', a)$  do
17 |           if  $(p, q) \notin R$  then
18 |              $R \leftarrow R \cup \{(p, q)\}$ ;
19 |              $worklist.enqueue((p, q))$ ;
20 return  $Q^2 \setminus R$ ;
```

3. Implementation

The library to which we have implemented this algorithm is called Mata². This library is being developed at BUT FIT at the VeriFit research group. Mata is a C++ fast and simple automata library, which supports many operations on NFAs including language concatenation, union, inclusion checking and more, including simulation computation, which is done using the RT algorithm[1].

3.1 Implementation phases

During benchmarking we have discovered multiple times that our implementation was not optimal. The implementation underwent six optimizations, with the fourth versions having a big time/space trade-off. We now present all of the versions and highlight their strengths and weaknesses, and we show a scatter plot that compares each version with the *RT* algorithm at the end of this report. We will refer to Algorithm 1 to show how we implemented or optimized different parts of the algorithm.

²<https://github.com/VeriFIT/mata>

3.1.1 First version

The initial implementation is very naive but it served as a good starting point for further optimizations. We have checked the correctness of the algorithm, which means that it produces the same simulation relation as the already implemented *RT* algorithm but with a different time complexity.

Computation of the reverse automaton (*line 2*) is done by calling the Mata `revert` function. The final simulation set (*line 4*) is implemented as

```
vector<pair<State, State>>,
```

which is the biggest weakness of this version, because the time needed to check pair membership, using the function `vec_contains`, is $\mathcal{O}(n)$.

The output of the simulation function must be in the structure

```
Simlib::Util::BinaryRelation
```

because the reduction mechanism implemented in Mata takes the simulation relation as an input in this structure.

The variable `result_sim_tmp`, that is represented as

```
vector<vector<bool>>>
```

is used to store the result relation and at the end is converted into the output structure.

Another weakness of this version is the data structure that was used to store the transition functions size (*line 6*). We have used a dynamically allocated two-dimensional matrix of integers, which was unnecessarily hard to maintain, so we have replaced it by the second version.

The rest of the implementation (*lines 5-20*) is straightforward, mainly copying all of the cycles and conditions from Algorithm 1.

3.1.2 Second version

Two big changes have been done in this version.

First is the removal of the matrix that has been used to store the transition functions size, which has been replaced with a one-dimensional array and an index function `index_fn`, which is used to access elements of the array as a matrix. The allocation of the matrix is simpler and work with it is easier.

The second change is the removal of the final simulation set and the `vec_contains` function. We have discovered that the set is redundant, because we already have a variable `result_sim_tmp` that keeps all the information we need and has access $\mathcal{O}(1)$ time complexity.

3.1.3 Third version

The internal representation of automata in Mata is done by a cleverly designed data structure. This structure uses ordered vectors to simplify searching [1]. The first two versions use a lot of build-in functions that do binary search, which takes a lot of time on big automata. We have managed to reduce the number of binary searches from 4 to 2, removing two searches (lines 6, 7) in the *initial refinement* phase.

This improved the time efficiency but it requires us to manage a helper look-up table to manage information about used symbols. Benchmarking showed that searching inside the look-up table is faster than doing binary searches in the main algorithm. The additional look-up table also increases the space complexity.

3.1.4 Fourth version

This version implements a big time/space trade-off. The look-up table has been replaced with two arrays of integers and bools. We have been able to do this, because the look-up table only stored an index (integer) and information about the usage (bool). Firstly we thought that this version would be a lot faster than the previous version, but we were surprised by the fact that this is actually slower than the third version.

3.1.5 Fifth version

We have further reduced the number of binary searches (line 13) from 2 to 1. This improved the time complexity by a small fraction.

3.1.6 Latest version

The latest version does not implement anything new, but it improves loop functionality which had a big impact on the time complexity. Also the code is cleaned up to make it more pretty. We show the overall improvement in Figure 6.

4. Benchmarking

We have concluded extensive benchmarking, which can be found in the supplementary material. We have used 26158 automata and compared the time needed to compute the simulation relation by the *RT* algorithm and *INY* algorithm. We have optimized our implementation 5 times and did the comparison for every version creating a scatter plot and computing some additional statistics. Some of the benchmarking results can be found at the end of this report.

5. Further improvements

We have optimized the implementation of Algorithm 1 to the best of our abilities, but as can be seen in the

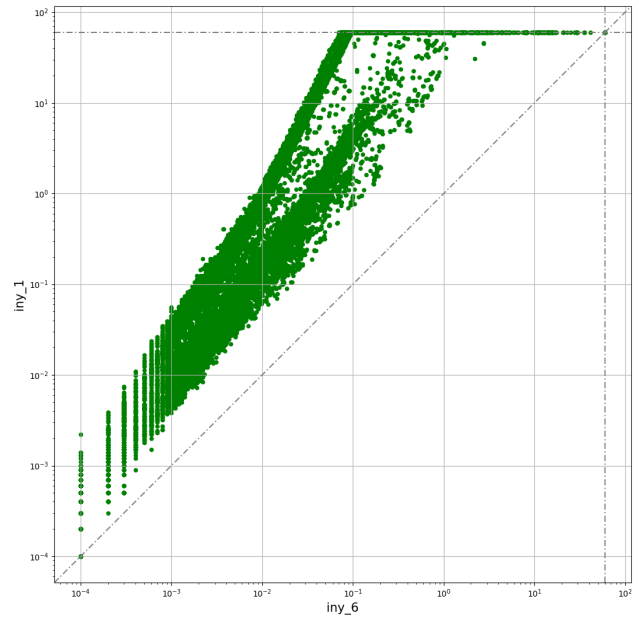


Figure 6. Version one compared to version six

scatter plots at the end of this report, we have not managed to surpass the implemented *RT* algorithm. Even with that in mind, we now present some knowledge we have gained during benchmarking and also give ideas on how to achieve an overall better reduction using the simulation relation.

5.1 initial refinement

At first we thought that refining the relation built in *initial refinement* (lines 5-9) would speed up the overall computation. This idea is not right because no matter how many states we add into the relation in *initial refinement*, we still need to loop through all of them in *propagation* (lines 10-19) and check for other possible simulations. This means that it does not matter if we add a relation in the *initial refinement* or *propagation* because if we add it in the first phase we then need to dequeue it at line 11 but that's the same as adding it to the worklist at line 19 and then dequeuing it again at line 11.

This is also closely related to another issue with the *INY* algorithm. The condition at line 7 contains only the first clause ($p \in F \wedge q \notin F$) which is not sufficient to build a proper simulation relation on NFAs. So while we can refine the initial relation as much as we want, we will not add any speed to the algorithm, in fact we will only slow it down because we must check more conditions, but we must have at least the two conditions that are specified on line 7 for the algorithm to work correctly.

5.2 Improved reduction

We propose an improvement not to simulation relation computation but to the reduction mechanism im-

Name	Max	Min	Median	Stddev	Mean	Wins (out of 26158)
Iny_1	60	0	0.2042	20.13800	9.67002	158
Iny_2	60	0	0.0842	5.28089	0.908176	210
Iny_3	60	0	0.0301	3.95896	0.512847	241
Iny_4	60	0	0.0315	4.11236	0.547181	230
Iny_5	60	0	0.0138	3.53319	0.409891	272
Iny_6	60	0	0.0085	3.13697	0.310584	490
Simlib_rt	60	0	0.0027	2.45819	0.137419	-

Figure 7. Table comparing different statistics

References

- [1] CHOCHOLATÝ, D., FIEDOR, T., HAVLENA, V., HOLÍK, L., HRUŠKA, M. et al. Mata: A Fast and Simple Finite Automata Library. In: *Tools and Algorithms for the Construction and Analysis of Systems*. Springer Nature Switzerland, 2024. ISBN 978-3-031-57249-4.
- [2] ESPARZA, J. and BLONDIN, M. *Automata Theory: An Algorithmic Approach*. 1st ed. The MIT Press, 2023. ISBN 9780262048637.
- [3] ILIE, L., NAVARRO, G. and YU, S. On NFA Reduction. In: *Theory Is Forever: Essays Dedicated to Arto Salomaa on the Occasion of His 70th Birthday*. Springer Berlin Heidelberg, 2004. ISBN 978-3-540-27812-2.
- [4] KOCOUREK, T. *IMPLEMENTATION OF ADVANCED SIMULATION IN THE MATA AUTOMATA LIBRARY*. Brno, 2024. Master's thesis. VUT FIT.
- [5] RANZATO, F. and TAPPARO, F. A New Efficient Simulation Equivalence Algorithm. In: *22nd Annual IEEE Symposium on Logic in Computer Science (LICS 2007)*. IEEE, 2007. ISBN 0-7695-2908-9.

plemented inside Mata. Currently only $\preceq_R$ is being computed and the reduction can be done only with R1. As we proved in Theorem 1 this means that we are not using the whole potential of the simulation relation because we can not use R3 to gain additional reduction. This could be a great and important subject for further work and improvements.

6. Experimental evaluations

On the last page you can find scatter plots comparing different versions of the *INY* implementation (referred to as *iny_x* on the horizontal axis) to the *RT* implementation (referred to as *simlib_rt* on the vertical axis). The units on the axes are seconds. In Figure 7 you can find different statistics we have gathered. Everything except the *wins* is in seconds. The *wins* in the *Simlib_rt* row are different when comparing with different *INY* implementations so we do not present it. You can see every version surpassed the previous except version 4.

7. Conclusion

In this report we have covered the basic theory of reducing NFAs, introducing equivalence as well as simulation reductions and proving that the reduction with simulations can be better than with equivalences.

We have introduced different versions of our implementation as well as shown the results of our extensive benchmarking efforts. Scatter plots and statistics at the end of this report show that every version, except version four, improved the overall results by little bit.

Although this implementation is far from successful, we believe that it has led us to discover new ways to further improve the potential overall reduction in Mata.

8. Thanks

I thank Ondřej Lengál for the opportunity to work on this project, for his endless patience with me and for the great advices he gave me on our weekly consultations. I also thank David Chocholatý because he gave me an excellent detailed introduction to Mata.

